

Deep Implicit Layers for Smooth Implied Volatility Surface Interpolation*

✉ Rafael Zimmer

Institute of Mathematical Science and Computing
University of São Paulo - Brazil
rafael.zimmer@usp.br

Abstract—We implement a neural network architecture based on deep implicit layers with the objective of optimizing the parameters of multiple SABR (stochastic alpha, beta, rho) models for interpolation and extrapolation of implied volatility surfaces in the context of financial options. The parameter values for the models are trained on daily data points and used for generating smooth interpolated points in the surface. The motivation behind our approach is to reduce arbitrage opportunities when generating smooth interpolation and extrapolation of the surface, i.e., making it infinitely differentiable at all points while still matching existing contracts. The proposed architecture solves the problem of selecting the candidates α, ρ, ν while minimizing statistical arbitrage opportunities for observed contracts within the respective generated surface. Our methodology includes using historical observed values of listed options as well as the calculated implied volatility for these contracts. The innate ability of transformer architectures to process observations of sequences with non-fixed lengths, i.e., for contracts with varying expiration dates is used to choose candidate points that best replicate the daily surfaces. Using our approach, the resulting surface curvature and volatility values are then compared to two baseline approaches: linear interpolations and non-implicit transformer networks and show either a smoother fit to the observed data or a reduction in arbitrage opportunities, respectively.

Index Terms—Neural Networks, Implied Volatility, Transformers, Quantitative Methods, Implicit Layers.

I. INTRODUCTION

A. Brief Introduction to Options and Implied Volatility

Option contracts are one of the most important types of derivative contracts in the context of financial markets. A derivative is a contract traded with respect to another asset (also called the underlying asset) and an option contract allows the buyer of the contract to sell or buy the specified asset from the seller at a predefined future date. The pricing equations for these derivatives, called the Black-Scholes model, require knowing the values for the time to maturity of the contract, the price of the underlying asset to be traded, the price of the asset to be traded and the market's risk-free rate at the time [1]. An additional variable, called the implied volatility (IV or σ_{iv}) is also required for the pricing of these contracts. This value refers to the market's expected value for the volatility of the asset price up until the time of maturity.

Having access to the implied volatility values allows investors to make sales at the prices consistent with the current expected contract price, that is, the no-arbitrage price

according to the market expectancy. However, the IV values corresponding to all possible strike prices and maturities are not directly observable in the market. Ensuring a way to obtain these values with confidence and readily available data in the market is a crucial task for quantitative analysts [12]. Usual strategies for estimating these values daily are predominantly based on linear or spline interpolation algorithms [9] or stochastic differential equation models [10] that use numerical methods to approximate the values from existing listed contracts. Both methods try to approximate the correct value for the implied volatility for each pair of time to maturity and strike price. This approach allows visualizing the model results in the form of a surface, where the $\mathbf{XY}$ axes are the maturity and strikes pairs and the $\mathbf{Z}$ axis is the IV . This surface, called the **implied volatility surface** (IVS), noted as $\mathcal{S}$ is therefore a function of the maturity time t and the strike price K ,

$$\mathcal{S} := f(t, K), \quad \text{where } f: \mathbb{R}^2 \rightarrow \mathbb{R}. \quad (1)$$

Ensuring accurate predictions of the IVS is crucial, as even small errors can lead to incorrect pricing and consequently large losses of profit or even negative return [8] given the leveraging capabilities of options when compared to non-derivative securities. Classic models such as linear interpolation, polynomial fitting, and *splines* not only lack flexibility for extrapolation, but also generate either non-smooth surfaces, arbitrage opportunities, or both [8].

B. Contributions of the Presented Work

In this paper, we propose an approach to address the challenges of interpolation and extrapolation of the IVS, focusing on reducing arbitrage opportunities. The chosen method uses a stochastic differential equation solution approach, specifically the *Stochastic Alpha, Beta, Rho* (SABR) model. The parameters α, ρ, ν of the SABR model are obtained through the Broyden-Fletcher-Goldfarb-Shanno (BFGS) quasi-Newton non-linear optimization algorithm [2, 4, 5, 19]. This step is commonly performed using only a subset of the points observed daily [13], due to the fact that using all points introduces various outliers that do not match the real prices of the derivative, which in turn increases the variance of the final model. To solve this difficulty, the points chosen for the optimization step are often manually filtered according to criteria set by human experts [6], and only a candidate

Thanks to the Brazilian Fundação de Amparo à Pesquisa do Estado de São Paulo (FAPESP) for the financial support.

subset of the original points is used. Rather than manually selecting these candidate points, the use of models capable of receiving point clouds and returning a score, such as transformers, to select the best points automatically has been proposed by Hagan et al. [6] instead. The main contribution of this research will be the adaptation of *neural implicit layers*, a physics-inspired variation of fully connected linear layers that outputs a solution to a fixed point equation, to be used as the output layer to a transformer architecture. This approach is an implicit fixed-point solver variation for the scoring model proposed by Kim et al. [13], aiming to enforce a zero arbitrage condition in the optimization process. This type of layer has efficiently been shown to perform well representing smooth surface densities and demonstrated state-of-the-art performance in function fitting tasks similar to the one presented here [11, 15], all while being computationally faster for solving interpolation tasks than linear layer models with similar performance [15]. Additionally, transformers have also been shown to perform better at handling unordered point clouds over time compared to recurrent neural layers [14].

C. The Transformer Architecture

The aforementioned *SABR* model is a stochastic volatility model used to obtain smooth surfaces, i.e., continuous and infinitely differentiable at all points [7]. In the context of this work, a score will be assigned for each candidate point of the daily listed options contracts, called the *summarized suitability* score (*SS* score) by Kim et al. [13]. This score corresponds to the suitability of the candidate point to summarize the characteristics of the generated surface, i.e., how feasible it is to reduce the number of candidates when optimizing the values for one of the parameters of the *SABR* model without altering the resulting surface shape or its smoothness¹. In summary, the score is used to select the highest valued points, and therefore the most representative of the surface, removing those that might introduce arbitrage in the optimization process for each of the *SABR* models [13] (a separate model is fitted for each unique strike price in a given day). Section I-D covers the details of fitting the *SABR* model at each strike price.

D. Deep Implicit Layers

Differently from feedforward layers in which the transformation from input to output is defined explicitly, implicit layers are instead defined by conditions that the output must satisfy. For explicit layers, a given output z is computed directly from the input x using a function f , such that $z = f(x)$. In contrast, an implicit layer is the function g so that it depends on both the input x and the expected output z , where z is a solution for the equation and the solution to the layer, namely the weights $\mathbf{W}$ are such that

$$g_{\mathbf{W}}(x, z) = 0. \quad (2)$$

¹The use of a per-point score is similar to what the principal components of a collection of points is when using the Principal Component Analysis (PCA) technique.

Formulating the problem as a fixed point equation can be used for a multitude of problems, including finding fixed points in recurrent networks, solutions to differential equations leading to Neural ordinary differential equations, and several physics optimization problems [3]. The implicit formulation offers several practical advantages, primarily the separation of the solution method from the layer definition which can be useful in maintaining interpretability of the resulting model [11] and as a way to enforce constraints in the optimization process.

Moreover, implicit layers offer significant benefits specifically in deep learning and automatic differentiation frameworks. Automatic differentiation methods store the entire computation graph, which can be memory-intensive. With implicit layers, however, the implicit function theorem allows computing gradients directly at the solution point and therefore storing intermediate variables becomes redundant [18]. This improves memory efficiency, making implicit layers particularly advantageous when substituting existing layers within large deep learning models [16].

E. Anderson Acceleration for Implicit Layers

The Anderson Acceleration (AA) method is a technique used to accelerate the convergence of fixed-point iterations. It is particularly useful when the fixed-point iteration is slow to converge or when the fixed-point iteration is used in a deep learning context. The method is based on the idea of extrapolating the current fixed-point iteration using a linear combination of the previous iterations. For this work, we use a Deep Equilibrium Model (DEM) variation of the Anderson Acceleration method, which is particularly useful for implicit layers in neural networks. The AA method is used to solve the fixed-point equation in the implicit layer, which is defined as the solution to the layer weights $\mathbf{W}$. It works by extrapolating the current fixed-point iteration using a linear combination of the previous iterations, as given by:

$$\mathbf{W}_{k+1} = \sum_{i=0}^m \alpha_i \mathbf{W}_{k-i} \quad (3)$$

where $\mathbf{W}_k$ is the current fixed-point iteration, $\mathbf{W}_{k-i}$ is the previous fixed-point iteration, and α_i are the coefficients of the linear combination.

The AA method is particularly useful for implicit layers in neural networks, as it allows for faster convergence of the fixed-point iteration. Our proposed architecture uses the AA method to solve the fixed-point equation in the implicit layer, which is defined as the solution to the layer weights $\mathbf{W}$. A pseudocode for our implementation of the AA method is:

Algorithm 1 Anderson Acceleration for Implicit Layers

```

function ANDERSONACCELERATION( $\mathbf{W}, m, k$ )
   $\alpha \leftarrow \text{SolveLinearSystem}(\mathbf{W}, m, k)$ 
   $\mathbf{W}[k+1] \leftarrow \sum_{i=0}^m \alpha[i] \cdot \mathbf{W}[k-i]$ 
  return  $\mathbf{W}[k+1]$ 
end function

```

II. IMPLEMENTATION OF A PARAMETETRIC SABR MODEL AND IMPLICIT NEURAL NETWORK ARCHITECTURES

A. Defining the SABR Model Equations

The SABR model is a stochastic differential model, defined by the following equations in which the dynamics of the forward price and the corresponding volatility levels are used to calculate the implied volatility.

- **Alpha** (α): This is the initial volatility level.
- **Beta** (β): The elasticity parameter between the volatility of the asset and its price.
- **Rho** (ρ): Statistical correlation between the asset price and its volatility, also called *spot-volatility correlation*.
- **Volvol** (ν): The volatility of the volatility process itself.

The implied volatility function of the SABR model is then given by:

$$\sigma_{BS}(f, K) = \alpha \frac{(fK)^{\frac{1-\beta}{2}}}{1 + \left(\frac{(1-\beta)^2 \rho^2}{24} + \frac{(1-\beta)^2 (2-3\rho^2)}{1920} \right) (fK)^{1-\beta}} \quad (4)$$

In the context of this paper, multiple SABR models are fitted daily, corresponding to each unique maturity value at the respective day. A continuous function is used to map the different maturities to the three model parameters (values for β are fixed and remain unchanged). The three variables used to encode the maturity levels are the following:

- **P**: Given a function $\alpha_t = f_\alpha(t, \mathbf{P}) \rightarrow \mathbb{R}$, the values for α for each individual
- SABR model are encoded by **P**, such that $\mathbf{P} \in \mathbb{R}^5$.
- **Q**: Given a function $\rho_t = f_\rho(t, \mathbf{Q}) \rightarrow \mathbb{R}$, the values for ρ for each individual
- SABR model are encoded by **Q**, such that $\mathbf{Q} \in \mathbb{R}^4$.
- **R**: Given a function $\nu_t = f_\nu(t, \mathbf{R}) \rightarrow \mathbb{R}$, the values for ν for each individual
- SABR model are encoded by **R**, such that $\mathbf{R} \in \mathbb{R}^4$.

And finally, the daily parametric SABR model is given by the set of encoded parameters $\{P, Q, R\}$ and the following equations to calculate values for $f_\alpha, f_\rho, e, f_\nu$ at each unique maturity value t :

$$f_\alpha(t, \mathbf{P}) = p_0 + \frac{p_3}{p_4} \left(\frac{1 - e^{-p_4 t}}{p_4 t} \right) + \frac{p_1}{p_2} e^{-p_2 t} \quad (5)$$

$$f_\rho(t, \mathbf{Q}) = q_0 + q_1 t + q_2 e^{-q_3 t} \quad (6)$$

$$f_\nu(t, \mathbf{R}) = r_0 + r_1 t^{r_2} e^{r_3 t} \quad (7)$$

The task of tuning the SABR model parameters is a supervised regression task. In machine learning, regression refers to problems where the output is a continuous variable, as opposed to classification, where the output is a discrete category. In our case, the output of the transformer model are the continuous SS scores for each parameter α , ρ , and ν . Then, the top $p = 80\%$ candidate points are passed as inputs to the BFGS algorithm, targeting the values **P**, **Q**, and **R** that minimize possible arbitrage opportunities of contracts by minimizing the

error function with respect to the selected candidate set. A list of the most commonly occurring strike and maturity pairs, and their corresponding implied volatilities are used for the error function calculation, similarly to the selection a human trader would make when selecting the most relevant data points to train the model [7, 13].

B. Proposed Encoder Architecture and Error Functions

1) *Monte Carlo Estimation of Z-Scores*: Transformers are a specific type of neural network architecture capable of capturing complex dependencies in sequential non-fixed length data. Within our proposed architecture, we use a transformer encoder to approximate the SS scores for each candidate point $S_i^t \in \mathcal{S}^t$ from the daily observed input. Then, for each training epoch it estimates a loss score for the mean squared error between the encoder's output and a z-score obtained by Monte Carlo estimation from a candidate set $\mathcal{S}^t$ of $k = 1e2$ generated subsets from the original set. This is done to calculate the estimated z-score of the point, which is finally used as the target value for our model. Consequentially, if after training the model has converged to a mean low error for the estimated scores, the network parameters are deemed optimized to substitute the Monte Carlo process in estimating daily SS scores for the currently listed option contracts [13].

Algorithm 2 An algorithm with caption

$i = 2$
 $a = 2$

2) *Details on the Chosen Encoder Architecture*: Unlike recurrent neural networks (RNNs) and convolutional neural networks (CNNs), transformers do not rely on a sequential structure to process data. Instead, they use an attention mechanism, which allows each input element (or token) to consider the relationship with all other elements in the same sequence simultaneously. This is achieved through a series of layers called encoders (for encoding the input data) and decoders (for generating the output).

We specifically use the self-attention mechanism, which accepts continuous data and calculates attention for different heads through value (value), key (key), and query (query) matrices, resulting in an output weighted by the importance of each part of the sequence [20, 22]. When dealing with self-attention, the three matrices are the same.

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V \quad (8)$$

Transformers are classified as machine learning algorithms due to their ability to learn patterns and relationships from data. They are trained using supervised techniques. In the context of the specific architecture we implemented, we use only the encoder block, with 4 input dimensions, 8 attention heads per block, and 8 blocks in total. The hidden values are passed to fully-connected (FC) layers, and a final implicit fixed-point iteration block is used to obtain the scores as SS z-normalized values. The model training stage is carried out

on historical option data listed for the **S&P 500** from January to December of 2021. The training epochs adjust the layer weights with an error function that minimizes the difference between its predictions and the Monte Carlo expected z-score values.

3) *Details on the Implicit Layer:* As defined in the previous chapter, we choose the implicit layers as the final hidden layers for our architecture for its summarization capabilities. We use the Anderson Acceleration framework [21] as suggested by [15] for NLP and other sequential models. We chose the Deep Equilibrium Model variation for the Implicit Layer, which means using the following fixed point equation instead of the one at Eq. 2

$$z^* = f(z^*, x) \quad (9)$$

The batched non-normalized Jacobian to be used as the gradient function in our code is given by the following equation, where z^* is the target output and f is our activation function. Finally, y are the hidden values as well as the final output normalized scores.

$$\left(\frac{\partial z^*(\cdot)}{\partial(\cdot)} \right)^T y = \left(\frac{\partial f(z^*, x)}{\partial(\cdot)} \right)^T g.$$

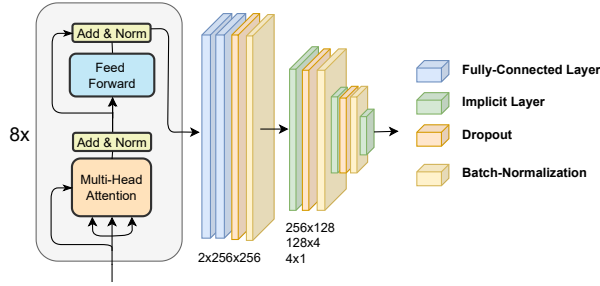


Fig. 1: Visualization of the Transformer Encoder and Implicit Fixed Point Iteration Layers

III. METHODOLOGY AND RESULTS

A. Data Description and Pre-Processing

The data used for training was obtained from historical options data on the *SPY* asset from 2021 to 2022. The dataset contains various columns about traded options, indexed by date, including strike prices, future prices, maturities, implied volatilities, among others. Relevant to our study, the principal columns used were:

- **Underlying:** Price of the underlying asset.
- **Strike:** Option's strike price.
- **Expiration:** Days remaining until the option's expiration.
- **Implied Volatility:** Value for the implied volatility.
- **Date:** Current date, used as the preprocessed set index.

Additionally, two columns were calculated to be used as features for the model:

- **Maturity:** Calculated by dividing *Expiration* by 252 (working days in a year) to normalize the maturity time.

- **Risk-free rate:** Risk-free rate obtained from the Federal Reserve Economic Data (FRED) for the corresponding period.
- **Dividend:** Dividend yield paid quarterly by the *SPY* index.

B. Data Preprocessing

The data preprocessing involved several essential steps, predominantly performed to allow the use of the data directly in the transformer architecture and subsequent use to generate the surface for historical data. Each of the trained transformers (one for each parameter α , ρ , or ν) takes four dimensions as input, which are:

- 1) Time to maturity;
- 2) The value of the parameter (α , ρ , or ν);
- 3) 1 if the parameter was calculated for the current day, 0 if it was calculated for a fixed set of maturities (top 7 most frequent maturities) using the previous day fitted parameters;
- 4) Parameter value calculated using the previous day's **P**, **Q** and **R** values for same strike and maturity.

The target variable is the scores generated using all candidate points for the current day, and the model's error function uses the percentage $p = 80\%$ of points and is trained to minimize the quadratic error function of the scores. For training and testing we also observed that the choice of $\beta = 0.5$ performed best at minimizing the error function.

C. Results and Surface Comparison

A baseline linear interpolation implementation for the daily observed points was also used to compare our trained model's performance by interpolating the points internal to a grid of strikes and maturity dates generated for within the boundary pairs for each day, that is, no extrapolation was performed due to the hardships of the linear model to predict values outside the observed range.

Our transformer model was trained on the dataset, with about 68 days and 44k observations total. With an early stopping sensitivity of $\epsilon = 10^{-3}$, the training was performed with a maximum epoch range of 10^3 epochs using the Adam optimizer. For the implicit layer, we used the Anderson Acceleration algorithm with a maximum of 10 iterations and a convergence threshold of 10^{-6} . The training time for the transformer model was approximately 8 minutes and 11 seconds, while the Monte Carlo preprocessing time was around 21 minutes and 5 seconds for the entire dataset. Due to the nature of the implicit layer, a simple batch size of 1 was used to avoid memory issues.

Model	Test Loss (RMSE)	Train time (Transformer)	Preprocess Time (Monte Carlo)
SST_α	1.21	~4m21s	~7m50s
SST_ρ	0.72	~2m8s	~6m34s
SST_ν	0.75	~1m43s	~6m41s

TABLE I: Model training metrics for each of the three SST models.

Separate tests using the trained model with and without the implicit layer were performed to compare the generated surface's error. The results of the training and testing of the model are shown in table I and section III-C. The RMSE values for the test data show that the model was able to generalize well to the test data, with the lowest RMSE value for the ρ parameter.

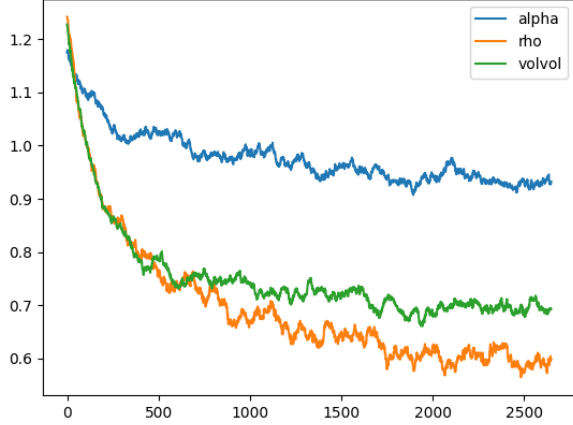


Fig. 2: Historical loss function from a Transformer training session on train data.

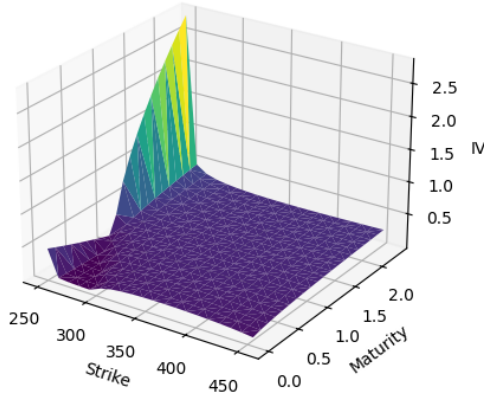


Fig. 3: Surface generated using a manually optimized SABR model. Notice how the non-removal of possible outlier candidates generates a out-of-range peak in the surface.

IV. CONCLUSION

A. Summary

This work presents an architecture taking as inspiration approaches from both Kim et al. [13] and Kolter et al. [15]

for the task of adjusting the parameters of the SABR model using a neural network based on the transformer architecture (specifically the *encoder* element) with implicit deep equilibrium layers as the regression component. The combination of the SABR model with the transformer architecture was used to replicate the patterns of the daily implied volatility surface, generating a smooth surface that reduces arbitrage opportunities for the existing contracts observed on the day. The candidate scoring and selection process is performed automatically, instead of manually, and the fine-tuning of the model to new daily observations is extremely fast as shown by comparing the Monte Carlo estimation method to the Model's training time in Table I. The possibility of using these models on modern GPU, which allows for faster model training times allow investors to fine-tune models and identify new trading opportunities in much smaller time intervals.

B. Implications and Future Work

The main focus of this work was to merge the approach using the transformer and the implicit layers to replace the manual selection of scores for candidates, as well to show the possibility of future research focusing on High-Frequency Trading (HFT) strategies options arbitrage strategies. Additional market factors and input features are also an interesting improvement to the chosen approach. Furthermore, it is important to consider the choice of the optimizer and subset sizes for the parameters $\mathbf{P}$, $\mathbf{Q}$, $\mathbf{R}$, as the time required for the pre-processing step is a major limitation. Adapting the model to other financial instruments is also a promising extension to the current work, as it allows for the study of the model's adaptation to different market regimes and economic conditions (both micro- and macroeconomic), which may reveal new trading opportunities.

REFERENCES

- [1] Fischer Black and Myron Scholes. The pricing of options and corporate liabilities. *Journal of Political Economy*, 81(3):637–654, 1973.
- [2] Charles G Broyden. The convergence of a class of double-rank minimization algorithms 1. general considerations. *IMA Journal of Applied Mathematics*, 6(1):76–90, 1970.
- [3] Ricky T. Q. Chen, Yulia Rubanova, Jesse Bettencourt, and David Duvenaud. Neural ordinary differential equations. 2019. URL <https://arxiv.org/abs/1806.07366>.
- [4] Roger Fletcher. A new approach to variable metric algorithms. *Computer Journal*, 13(3):317–322, 1970.
- [5] Donald Goldfarb. A family of variable-metric methods derived by variational means. *Mathematics of Computation*, 24(109):23–26, 1970.
- [6] Patrick Hagan, Deep Kumar, Andrew Lesniewski, and Diana Woodward. Managing smile risk. *Wilmott Magazine*, 1:84–108, 01 2002.
- [7] Patrick Hagan, Andrew Lesniewski, and Diana Woodward. *Probability Distribution in the SABR Model of Stochastic Volatility*, volume 110, pages 1–35. 06

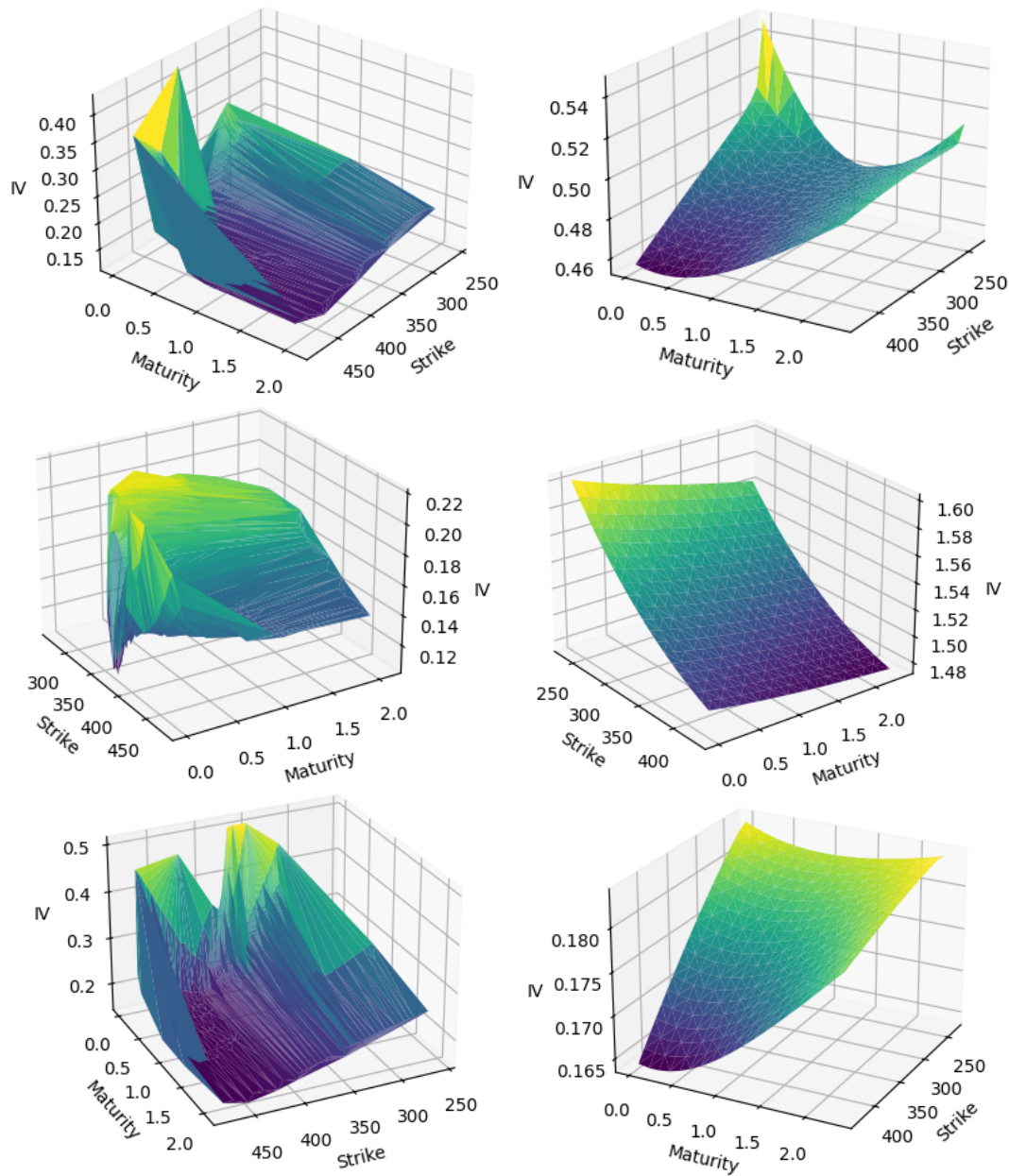


Fig. 4: Surfaces interpolated on test data using a linear interpolation model (left) compared to the SABR model fitted with the output of the implicit transformer model (right).

2015. ISBN 978-3-319-11604-4. doi: 10.1007/978-3-319-11605-1_1.
- [8] Patrick S. Hagan, Deep Kumar, Andrew Lesniewski, and Diana Woodward. Arbitrage-free sabr. *Wilmott*, 2014(69):60–75, 2014. doi: <https://doi.org/10.1002/wilm.10290>. URL <https://onlinelibrary.wiley.com/doi/abs/10.1002/wilm.10290>.
- [9] Cristian Homescu. Implied volatility surface: Construction methodologies and characteristics, 2011.
- [10] Richard Jordan and Charles Tier. Asymptotic approximations to cev and sabr models. *SSRN Electronic Journal*, pages 1–38, May 2011. Available at SSRN: <https://ssrn.com/abstract=1850709> or <http://dx.doi.org/10.2139/ssrn.1850709>.
- [11] Kenji Kawaguchi. On the theory of implicit deep learning: Global convergence with implicit layers. 2021. URL <https://arxiv.org/abs/2102.07346>.
- [12] Bo-Hyun Kim, Daewon Lee, and Jaewook Lee. Local volatility function approximation using reconstructed radial basis function networks. In J Wang, Z Yi, JM Zurada, BL Lu, and H Yin, editors, *ADVANCES IN NEURAL NETWORKS - ISNN 2006, PT 3, PROCEEDINGS*, volume 3973 of *Lecture Notes in Computer Science*, pages 524–530, HEIDELBERGER PLATZ 3,

- D-14197 BERLIN, GERMANY, 2006. Univ Elect Sci & Technol China; Chinese Univ Hong Kong; Asia Pacific Neural Network Assembly; European Neural Network Soc; IEEE Circuits & Syst Soc; IEEE Computat Intelligence Soc; Int Neural Network Soc; Natl Nat Sci Fdn China; KC Wong Educ Fdn, Hong Kong, SPRINGER-VERLAG BERLIN. ISBN 3-540-34482-9. 3rd International Symposium on Neural Networks (ISNN 2006), Chengdu, PEOPLES R CHINA, MAY 28-31, 2006.
- [13] Hyeonuk Kim, Kyunghyun Park, Junkee Jeon, Changhoon Song, Jungwoo Bae, Yongsik Kim, and Myungjoo Kang. Candidate point selection using a self-attention mechanism for generating a smooth volatility surface under the sabr model. *EXPERT SYSTEMS WITH APPLICATIONS*, 173, JUL 1 2021. ISSN 0957-4174. doi: 10.1016/j.eswa.2021.114640.
 - [14] Soohan Kim, Seok-Bae Yun, Hyeong-Ohk Bae, Muhyun Lee, and Youngjoon Hong. Physics-informed convolutional transformer for predicting volatility surface. *QUANTITATIVE FINANCE*, 24(2):203–220, JAN 9 2024. ISSN 1469-7688. doi: 10.1080/14697688.2023.2294799.
 - [15] Zico Kolter, David Duvenaud, and Matt Johnson. Deep implicit layers - neural odes, deep equilibrium models, and beyond, 2020. NeurIPS 2020 tutorial. Retrieved from <https://implicit-layers-tutorial.org/>.
 - [16] Xuechen Li, Ting-Kam Leonard Wong, Ricky T. Q. Chen, and David Duvenaud. Scalable gradients for stochastic differential equations. 2020. URL <https://arxiv.org/abs/2001.01328>.
 - [17] M. Ludkovski. Statistical machine learning for quantitative finance. *ANNUAL REVIEW OF STATISTICS AND ITS APPLICATION*, 10:271–295, 2023. ISSN 2326-8298. doi: 10.1146/annurev-statistics-032921-042409.
 - [18] Stefano Massaroli, Michael Poli, Jinkyoo Park, Atsushi Yamashita, and Hajime Asama. Dissecting neural odes. 2021. URL <https://arxiv.org/abs/2002.08071>.
 - [19] David F Shanno. Conditioning of quasi-newton methods for function minimization. *Mathematics of Computation*, 24(111):647–656, 1970.
 - [20] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. 2023. URL <https://arxiv.org/abs/1706.03762>.
 - [21] Homer F. Walker and Peng Ni. Anderson acceleration for fixed-point iterations. *SIAM Journal on Numerical Analysis*, 49(4):1715–1735, 2011. doi: 10.1137/10078356X.
 - [22] Qingsong Wen, Tian Zhou, Chaoli Zhang, Weiqi Chen, Ziqing Ma, Junchi Yan, and Liang Sun. Transformers in time series: A survey. 2023. URL <https://arxiv.org/abs/2202.07125>.